AI that computes with time and is already winning

A new family of learning machines, built for a world that happens as events, beats the best published models on public benchmarks with a fraction of their size.

5 of 5
datasets won on the public EasyTPP event-prediction leaderboard

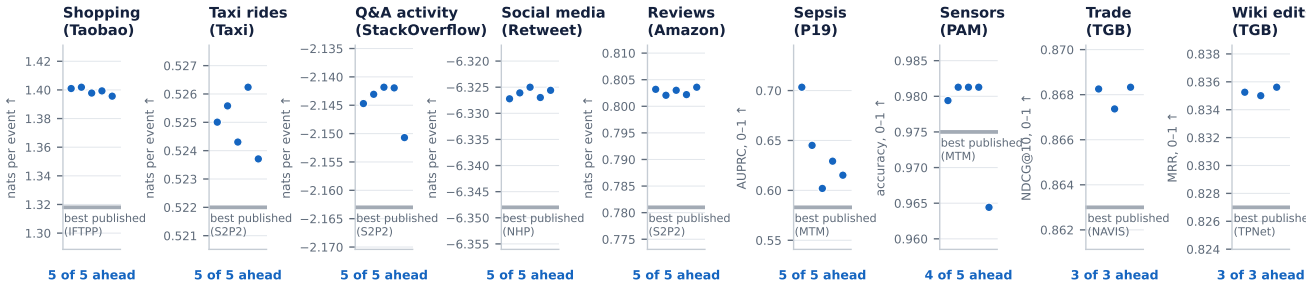
0.639
sepsis early-warning AUPRC vs 0.583 for the best published model ↑

15×
smaller and cheaper than the state-of-the-art model on Retweet, while more accurate

40 / 41
independent runs ahead of the best published result on the nine benchmarks won

In one sentence. Sleeping Machines is building a general-purpose computing substrate for machine intelligence, one that learns through time and sparse events, and its first public results are wins against the strongest published models, achieved by small, efficient models under strict, pre-registered test protocols.

40 of 41 independent runs ahead of the best published result



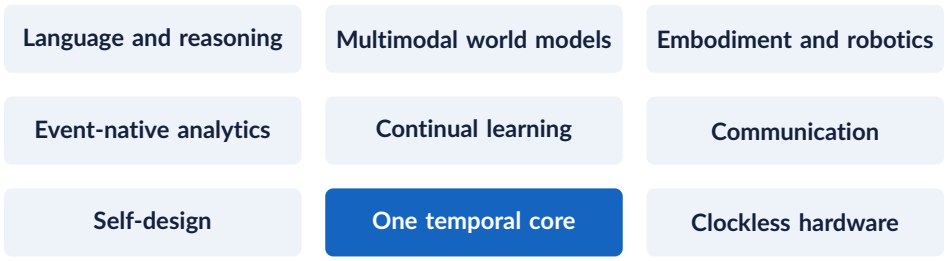
The benchmark wins use completed runs on official data, scored once on sealed test sets. The forward-credit development page reports separate private TRAIN-tail experiments, with its scope stated beside the numbers. Each benchmark's unit, and whether higher or lower is better, is stated beside every comparison (↑ higher is better, ↓ lower is better). Sources: `experiments/B1_EASYTPP.md`, `experiments/B2_IRREGULAR_TS.md`, `report/I_SCIENCE.md`.

THE AMBITION

One substrate for machine intelligence

Today's AI runs on a clock: every input is chopped into fixed steps, and the whole model works at every step whether or not anything happened. The real world is not like that. Purchases, heartbeats, sensor readings, messages, machine logs and words arrive as *events*, irregularly, with meaning in their timing and in their silences.

Sleeping Machines builds learning machines in which **time itself does part of the computation**. Our goal is one trainable substrate that spans:



What makes it different, in plain terms:

- **Memories that live through silence.** Each memory keeps evolving between events, so "how long since" and "what happened in between" are information, not missing data.
- **A race of clocks decides what happens next.** Possible next events race against each other; the one that fires first wins. This gives exact predictions of both *when* and *what*, and the same mathematics reproduces the attention mechanism inside today's large language models.
- **Only the relevant parts wake up.** Updates are sparse and addressed: the machine can hold far more capacity than it uses for any single event, which is where the name comes from.
- **Learning from roads not taken.** The model is also credited for the alternatives that lost the race, so it learns which routes to take.

> **Why this matters commercially.** Most of the world's valuable data is timestamped events: transactions, clinical records, telemetry, security and IT logs, user behaviour. Models that understand time natively can be more accurate on that data and much cheaper to run, on CPUs and edge devices as well as in datacentres. The same principles point to hardware without a global clock, where idle capacity costs almost nothing.

THE PROVING GROUND

The benchmarks, in plain language

We compete on established public benchmarks, using their official data splits and scoring rules, against the best results other teams have published. The headline event benchmark is **EasyTPP** (ICLR 2024), the standard test suite for predicting *when* the next event happens and *what* it will be. Its current leader is **S2P2** (NeurIPS 2025, GE HealthCare and UC Irvine), a modern state-space model. The **Temporal Graph Benchmark** (TGB, NeurIPS 2023) tests who interacts with whom, and when, against temporal graph neural networks.

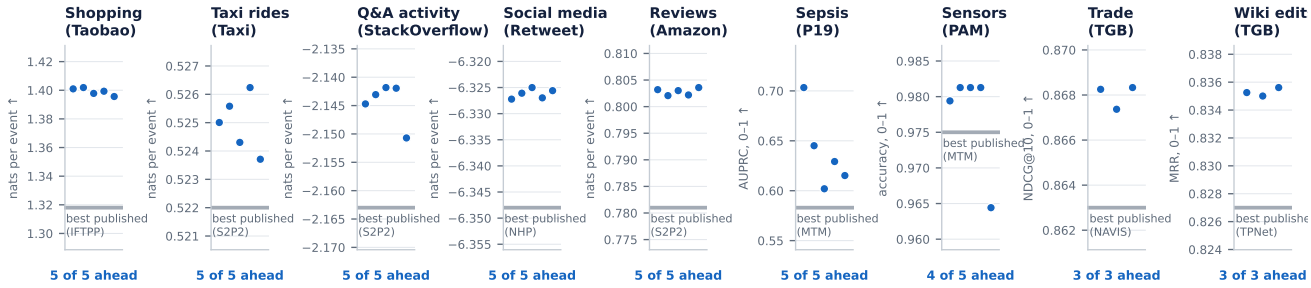
Benchmark	What the data is	Why it matters	Score (unit, direction)
Taobao	Clicks and purchases of users on Alibaba's Taobao marketplace, by product category	E-commerce: anticipating what a shopper does next and when	log-likelihood, nats per event ↑
Taxi	New York taxi pick-ups and drop-offs across city districts	Mobility and logistics: demand and fleet planning	log-likelihood, nats per event ↑
StackOverflow	Badges earned by users of the StackOverflow Q&A site	User engagement and lifecycle prediction	log-likelihood, nats per event ↑
Retweet	Retweet cascades on Twitter, by type of user retweeting	Social media: how information spreads, and how fast	log-likelihood, nats per event ↑
Amazon	Product reviews by Amazon customers, by category	Retail behaviour across product lines	log-likelihood, nats per event ↑
P19 sepsis	PhysioNet 2019: 38,803 intensive-care stays, 34 vital signs and lab values measured at irregular times	Early warning of sepsis, a leading cause of hospital deaths; every hour of earlier treatment matters	AUPRC and AUROC, 0 to 1 ↑
PAM	PAMAP2 physical-activity monitoring: 5,333 recordings from wearable inertial sensors (17 channels), 8 activities	Wearables and health monitoring: recognising what a person is doing from body-worn sensors	accuracy and F1, 0 to 1 ↑
P12 mortality	PhysioNet 2012: 11,988 intensive-care stays, 36 irregular measurements	Predicting in-hospital mortality from the first 48 hours	AUROC and AUPRC, 0 to 1 ↑
TGB trade	UN trade flows between 255 countries, 1986–2016	Which partners a country trades with most next year	NDCG@10, 0 to 1 ↑
TGB wiki	157,474 timed Wikipedia edits by 8,227 users on 1,000 pages	Which page a user edits next	MRR, 0 to 1 ↑
Neural-TPP benchmark (TMLR 2023/2025)	Seven event datasets (LastFM listening, MOOC course activity, Github, Stack Overflow, Wikipedia edits, MIMIC2 hospital visits, Retweets), five fixed splits each	A second, independent event benchmark: when the next event happens and what it is	negative log-likelihood, nats per sequence ↓

Reading the scores.

- **Log-likelihood (nats per event), higher is better.** It measures how much probability the model gave to what actually happened, both its timing and its type. A gain of 0.08 nats per event means the model assigns about 8% more probability to every real event, on average, compounding over every event in a sequence. Values can be negative; less negative is better.
- **AUPRC, higher is better (0 to 1).** How well the model ranks the rare patients who develop sepsis (4% of stays) ahead of everyone else, with emphasis on raising true alarms without false ones. It is the clinically relevant metric when the condition is rare. **AUROC** (0 to 1, higher is better) measures overall ranking.
- **Negative log-likelihood per sequence, lower is better.** The same probability measure summed over a whole sequence and negated, as the neural-TPP benchmark reports it; time and type ("mark") parts are reported separately and add up to the total.
- **NDCG@10 and MRR, higher is better (0 to 1).** How well the model ranks the true next partners or pages near the top of a list of all candidates.
- **Size and cost, lower is better.** Parameters measure how big a model is; multiply-adds per event measure the work needed for each prediction, which drives running cost and energy.

THE WINS

Ahead of the best published models



Each blue dot is one independent run (EasyTPP: five training seeds; sepsis and wearables: the five official data splits; temporal graphs: three pre-registered seeds), scored once on the sealed test set. The grey line is the best published result. Higher is better in every panel: log-likelihood in nats per event for the EasyTPP datasets, AUPRC (0 to 1) for sepsis, accuracy (0 to 1) for wearables, NDCG@10 and MRR (0 to 1) for the two temporal-graph benchmarks.

Benchmark	Best published	Sleeping Machines	Margin	Our size and compute vs the leading model
Taobao nats/event ↑	1.318 (IFTPP)	**1.399 ± 0.003**	+0.081	0.92× S2P2's compute per event; best time and type predictions
Taxi nats/event ↑	0.522 (S2P2)	**0.525 ± 0.001**	+0.003	**1/12 of S2P2's size and compute**; reproduced independently on separate hardware
StackOverflow nats/event ↑	-2.163 (S2P2)	-2.144 ± 0.004	+0.019	1.26× compute; a model of S2P2's size also wins (-2.153)
Retweet nats/event ↑	-6.348 (NHP)	-6.326 ± 0.001	+0.022	**1/15 of S2P2's size and compute**; S2P2 scores -6.365
Amazon nats/event ↑	0.781 (S2P2)	**0.803 ± 0.001**	+0.022	**0.29× S2P2's compute**, 3.6× smaller; best time and type predictions
Wearables, PAM accuracy ↑	0.975 (MTM)	**0.978 ± 0.007**	+0.003	**19× smaller** than MTM (46,316 vs 873K parameters); F1 0.980 vs 0.976; 4 of 5 splits ahead
Sepsis, P19 AUPRC ↑	0.583 (MTM)	**0.639 ± 0.039**	+0.056	62,681 parameters; AUROC 0.916 vs 0.903 ↑; without its temporal memory the same network drops to 0.572 on every split
Trade graphs, TGB tgbn-trade NDCG@10 ↑	0.863 (NAVIS)	**0.868 ± 0.0005**	+0.005	2,107 parameters (NAVIS 1,280); 3 of 3 sealed seeds ahead; a new domain
Wikipedia edits, TGB tgbl-wiki MRR ↑	0.827 (TPNet)	**0.835 ± 0.0003**	+0.008	7,995 parameters; 3 of 3 sealed seeds ahead; 4.5 ms per query on one CPU thread

± is the standard deviation across the runs (five; three on TGB). Compute is multiply-adds per event, counted from S2P2's released code at its published configuration.

One model for the whole benchmark. A single configuration of our model, with no per-dataset tuning, is ahead of the best published result on all five EasyTPP datasets: Taobao 1.397 vs 1.318, Taxi 0.526 vs 0.522, StackOverflow -2.144 vs -2.163, Retweet -6.324 vs -6.348, Amazon 0.802 vs 0.781 (nats per event ↑; 24 of 25 runs ahead; pre-registered protocol), at 0.07× to 1.03× the leading model's compute. Across all nine benchmarks, 40 of 41 independent runs are ahead.

A SECOND EVENT BENCHMARK

The same model, unchanged: the strongest type predictions, no total win yet

To test transfer, we took the frozen one-configuration EasyTPP model to the neural temporal-point-process benchmark of Bosser & Ben Taieb (TMLR 2023, 2025): seven datasets, five fixed splits each, every split scored once on its sealed test set. No setting was tuned for these datasets. The pre-registered bar is the best published time component plus the best published type component, a combination that no single published model reaches.

Dataset nats/sequence ↓	Bar	Sleeping Machines	Time / type parts (ours vs best published)	Verdict
Wikipedia edits	-122.62	$+11{,}421.66$	$+11{,}394.01$ vs -267.41 / **27.65 vs 144.79**	Behind: splits 1–4 average -277.54, but the guarded rerun of split 0 assigns exploding hazards to two test sequences; type part far ahead
MOOC course activity	-239.7	-226.85 ± 2.83	-298.57 vs -310.6 / 71.73 vs 70.9	Behind; timing gap
Stack Overflow	11.9	12.714 ± 0.874	-91.60 **vs** -91.1 / 104.31 vs 103.0	Behind; time part ahead
Github	-272.90	-270.62 ± 48.69	-390.93 **vs** -382.4 / 120.30 vs 109.5	Behind by 2.3; time part ahead; below every single published model's total
MIMIC2 hospital visits	2.42	7.01 ± 0.25	2.99 vs 0.13 / 4.02 vs 2.29	Behind
Retweets	-538.70	-515.16 ± 3.24	-598.81 vs -621.33 / 83.65 vs 82.63	Behind; type part level, timing gap
LastFM listening	-849.65	-774.64 ± 48.38	-1380.05 **vs** -1363.78 / 605.41 vs 514.13	Behind; time part ahead; below every single published model's total

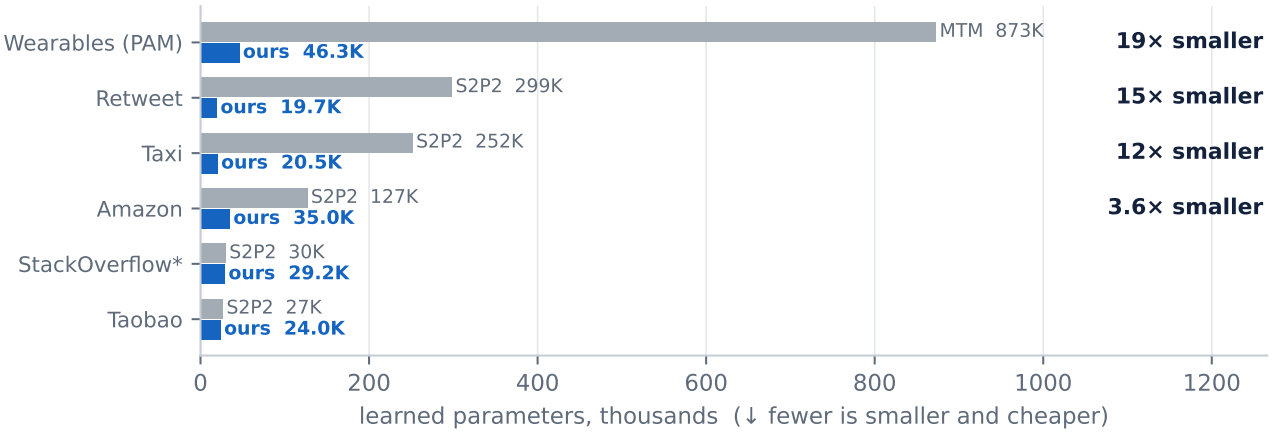
$\pm$ is the standard error over the five fixed splits. Negative log-likelihood per sequence: lower is better.

What Wikipedia shows. About 90% of consecutive edits return to the page just edited, and 10–27% of test pages never appear in training. Our addressed memory keeps one slot per page, written when that page occurs and read by that page's own score, so it can copy any page seen earlier in the sequence: type NLL 27.65 against 144.79 for the best published model, whose GRU scores pages as if guessing uniformly. The total is lost: one split's training failed numerically, and its pre-registered guarded rerun gives two test sequences hazards so confident that ordinary silences cost millions of nats. Bounding hazards is the development fix.

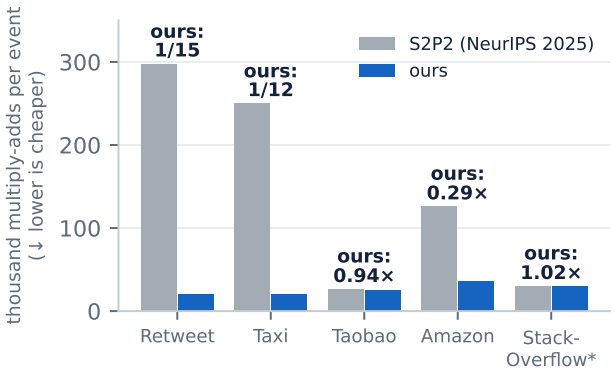
What the losses teach. A simple per-sequence counter, with no training, predicts the next type better than our model on two of these datasets, so the next development step adds persistent per-sequence type statistics to the memory. Github's training produced invalid values in four of five runs (one bad update poisoned the optimiser); those splits, and Wikipedia split 0, are retrained once with a guard that skips such updates, configuration unchanged, and scored once on test. The type-statistics step is development outside the sealed protocol.

EFFICIENCY

Winning while small



Learned parameters, in thousands; fewer means a smaller, cheaper model (lower is better). Grey: S2P2, the leading published state-space model on EasyTPP, at its published configuration for each dataset; for PAM, MTM (2025) at the parameter count it reports for PAM. Every Sleeping Machines model shown is also more accurate than the model beside it. *StackOverflow: the model matched to S2P2's size, which also wins.



Work per prediction, in thousands of multiply-adds per event (lower is cheaper). "ours: 1/15" means our model needs one fifteenth of S2P2's work.

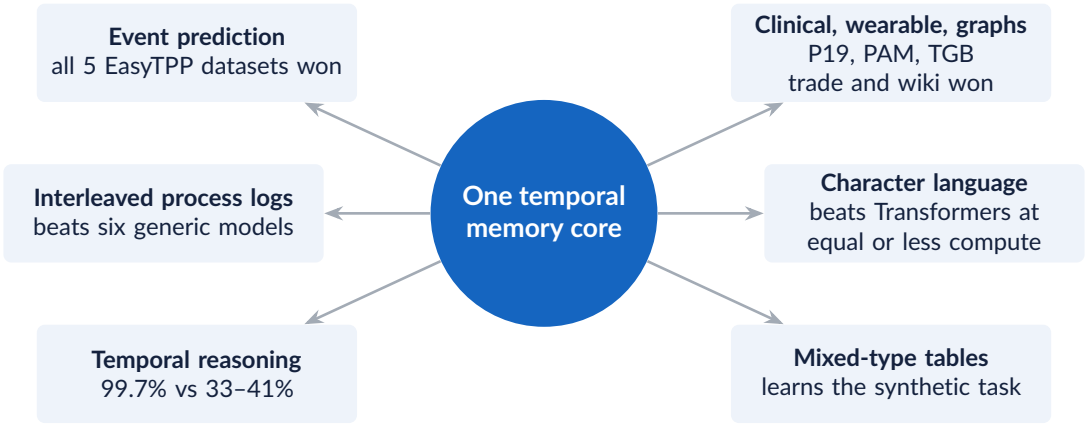
What size buys.

- **Running cost and energy** scale with the work per prediction.
- **Deployment anywhere**: models of 20,000 to 60,000 parameters run on ordinary CPUs, phones and embedded devices, close to the data.
- **Privacy**: clinical and industrial data can stay on site.
- **Headroom**: accuracy at small size means the architecture, not brute scale, is doing the work.

GENERALITY

One family across different kinds of data

S2P2 is shown on one kind of task. Our family wins on four kinds of data (event streams, clinical records, wearable sensors, interaction graphs) with shared machinery: the sepsis classifier reuses *the same temporal memory layer, same code and same size*, as the event-prediction model.



One shared core across four domains (development data, first seed). We trained a *single* temporal memory of 14,080 parameters on taxi rides, online shopping, Q&A activity and product reviews at the same time; only the small input and output layers differ per dataset. It matched or beat the four separately trained models on three of the four (log-likelihood, nats per event, higher is better: Taxi 0.490 vs 0.487, Taobao 1.290 vs 1.284, StackOverflow -2.186 vs -2.177), with about 40% fewer parameters in total. Mobility, shopping and online-community timing share one learned temporal dynamics. Amazon trailed and was still improving when training stopped; per-dataset training budgets are the next test.

Why generality matters for value. A company with one specialised model has one product. A company with a core that transfers across data types can serve many markets from one technology, and every new domain makes the core stronger.

WHY THE LARGER VISION IS ACHIEVABLE

Each layer of evidence supports the next

Next: scale and transfer. Large-scale language on GPUs, skill transfer between data types, clockless hardware.

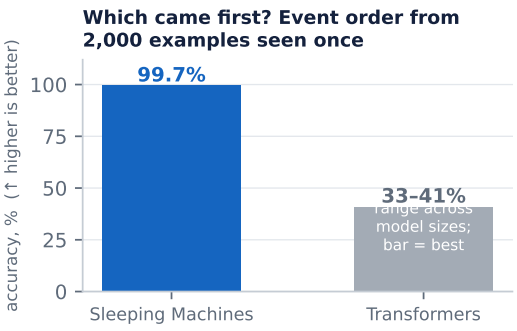
5. It generalises. One shared core across four event domains; one memory layer for events and clinical records.

4. It wins public benchmarks. All 5 EasyTPP datasets, P19 sepsis, PAM wearables and two TGB graph leaderboards, at up to 15× smaller size; 40 of 41 runs ahead.

3. It learns language efficiently. On 10M characters it beats a Transformer: 1.888 vs 1.908 bits per character ↓ with 0.40× its training compute. At 90M characters it beats a tuned Transformer-256×4 (1.783 vs 1.811 ↓) at 1.06× its compute.

2. The mechanisms work. Event order learned from 2,000 examples seen once: 99.7% accuracy vs 33–41% for Transformers ↑. Timing-only discrimination: 95.3% where any order-only model is capped at 50% ↑. Learning from roads not taken improves language 2.507 → 2.371 bits per character ↓ for 0.3% extra training work; more idle capacity improves it further at the same active work.

1. Mathematics: it contains today's leading architectures. A race of clocks reproduces the attention of Transformers *exactly*, and Mamba-style selective state-space models are exact members of the family (verified numerically to 4×10^{-13}). Everything those models can represent, ours can, plus native time and sparsity.



Why the steps add up. The family is not a narrow trick for one benchmark. Its mathematics contains the architectures that power today's AI; its mechanisms solve temporal problems those architectures fail at; it already learns language competitively at small scale; and it wins public benchmarks on four kinds of data with one shared memory design. Scale is the remaining work, and each layer beneath it is already demonstrated.

Hardware. In our cost model, the native language model moves 5.8× fewer bytes per character than a comparable Transformer at better quality (modelled, not yet measured on silicon; a funded milestone).

RIGOR

How we know the results are real

Sealed tests, rules fixed in advance. Every win uses the benchmark's official data splits. The reporting rule is written down before the test set is touched, and each model is scored on the test set exactly once.

The benchmark's own scorer agrees. Re-scoring our models with EasyTPP's own estimation method gives the same results within its sampling noise.

Independent runs. Benchmark headline results are averages of independent runs (five seeds or official splits; three pre-registered seeds on TGB), with its spread: 40 of 41 runs are ahead of the best published result.

We found and closed a scoring pitfall. Timestamps in these datasets are rounded (for example to whole seconds). Flexible models can exploit the rounding for large, fake gains. We built an audit that removes the rounding; we withdrew our own early results that failed it, and every reported win passes it.

Reproduced on separate hardware. The Taxi win was reproduced independently on a separate desktop computer with separately downloaded data: 0.5252 vs 0.5250 nats per event ↑, every run above S2P2. One-command kits let third parties rerun the EasyTPP and both TGB wins.

Losses reported as plainly as wins. Where we are behind, we say so with the numbers and the next test (next page). Comparisons always use the same units and protocol as the published result.

LEARNING MORE ECONOMICALLY

Teaching the model how to learn

The earlier pages show that our models can make strong predictions with small models. We are also tackling the cost of *learning*: deciding which parts of a model should change when new information arrives.

A sparse model takes only a few routes through its available memory and computation. To improve, it needs to learn from those choices, including alternatives it did not take. Trying every alternative can make training expensive even when prediction is cheap.

> **Our approach.** Use what the model discovers while making predictions to help it evaluate alternatives and choose better learning updates. The ambition is a model that learns both *what to predict* and *how to improve*, using the same event-based architecture.

98.4%
less error when estimating the value of alternative learning routes

2.09×
faster evaluation of alternatives in the measured experiment

77.8%
less randomness in the route controller's learning signal

What these results mean.

- **More accurate guidance.** On eight new language samples, information from the model and its learning state estimated the relative value of alternative routes more accurately than a fixed estimate. Every sample improved.
- **Less repeated work.** Reusing this information, with an exact correction, took 12.9 seconds instead of 26.9 seconds for separately evaluating all alternatives. Both measurements include discovering the alternatives.
- **A steadier learning signal.** Random route selection introduces noise into learning. In a separate controller experiment, the corrected signal reduced that source of noise while preserving its average direction.

Significance. These experiments demonstrate that information already produced by a model can make an expensive part of learning more accurate and more economical. They give a concrete engineering basis for our learning architecture. The measured speedup applies to evaluating learning alternatives, not to training an entire model.

Learning rules can themselves improve. In a separate event-prediction experiment, a trained controller adjusted the model's learning updates and improved prediction quality in all nine tested combinations of model depth and initialization. Those fits used more CPU time. This shows that learned update rules can improve quality within our model family.

Status: a working research prototype. The 98.4% result uses eight private language samples; speed uses four samples and three timing repeats; noise uses four samples and a fixed controller. These are separate component measurements. The update-controller result uses three initializations at depths 2/4/8 on Taxi training/development data and retains ordinary error propagation. Detailed methods and completed result hashes are in `investment/forward_credit_evidence_20261011.json`.

LEARNING WHILE THE MODEL IS IN USE

Learned guidance improves online forecasts

Markets produce a stream of transactions. A useful model has to learn as that stream arrives. Our aim is to improve both its predictions and the machinery that decides how new experience changes the model.

> **What we tested.** We trained an event-based credit model on earlier Bitcoin trades. It uses its own persistent memory and observed errors to decide when to apply credit across the predictor's layers. Its training signal is the forecast quality of actual updates on subsequent observations.

The experiment used real Binance trades. Each forecast predicts the next transaction's timing and type, and is scored before learning from the outcome. All rules start from the same acquired predictor and optimizer state. The comparison covers 16,384 transactions.

Learning during use	Prediction loss ↓
Our temporal traces + previously trained guidance	**5.737391**
Our temporal traces alone	5.737492
Random guidance, same number of credit adjustments (mean)	5.737596
Our temporal traces + guidance with online weight updates	5.737637
Our temporal traces + always-on effects across layers	5.737697
Local updates without temporal credit	5.737748
Ordinary online backpropagation	5.738462
Fixed model weights; observed memory still updates	5.752528

What we achieved. Previously trained guidance wins prediction quality by 0.000101 nats per transaction over temporal traces alone, and by 0.001071 over ordinary backpropagation. It beats all three random schedules using the same 57 credit adjustments; the mean gap is 0.000205 nats per transaction. These are small, measured gains in development forecasting.

Why it matters. A learning function trained on earlier data transferred a useful credit policy to later observations. The forward model stays the same; the learned guidance changes how it learns. The random comparison supports useful selection timing beyond occasionally applying extra credit.

How it adapts. Guidance weights stay fixed in the winning run, while its event-based memory continues to evolve. It applies extra credit on 37, 9, 7 and 4 updates in the four successive stream segments. Updating its weights online produced worse forecasts in this experiment.

Asynchronous-chip relevance. A separate early-request version improves forecasts over our temporal traces (5.737456 vs 5.737492), while computing remote credit in 257 of 512 windows. Remote matrix work falls **49.8%**, from 1.762 to 0.884 billion FLOPs, avoiding at least 5.745 million credit-state read/write scalar accesses. This counts one component; guidance, teaching and optimizer work are additional. Native races use local readiness joins; learning retains 32-target update boundaries. CPU timing describes the prototype, not asynchronous-chip advantage.

One restored 18,164-parameter predictor and one 31,753-parameter guidance initialization on reused 25 August development data; sealed test untouched. Earlier 24 August acquisition preserves the same actor and optimizer trajectory. Guidance preparation costs 28.3 seconds; original predictor fitting costs 5,750.7 seconds. Random schedules use three predeclared seeds, with activity calibrated from this development run (mean5.737596, sampleSD0.000069). Actor updates every32targets; guidance is trained by delayed actual-update utility, not gradient imitation. Forecasting, not trading profits. Completed methods, costs and hashes:

`investment/forward_credit_evidence_20261011.json`.

WHAT THE EVIDENCE ADDS UP TO

A proven prediction engine and a developing learning engine

Sleeping Machines aims to be a general-purpose substrate for language and reasoning, multimodal world models, embodiment, continual learning, communication, self-design and hardware. Its evidence already spans several parts of that ambition.

Strong prediction with small models. The public benchmark results lead this brief because they are the strongest evidence: nine benchmark wins across four kinds of data, including all five datasets of the standard event benchmark, with models up to 15 times smaller. That gives the model family a measured advantage on specific tasks, using existing computers.

A practical route to improving learning. We have measured more accurate and faster evaluation of learning alternatives, better predictions from learned update rules, and a small real-market prediction advantage from learning guidance that adapts during use. Together they show working pieces of a machine that learns how to learn. Each result has its own scope; the pieces are not one combined speedup.

A mathematical foundation. We have derived and numerically checked how route-level learning errors depend on prediction uncertainty and differences between alternatives. The direct route formulation can avoid multiplying local estimation errors through successive layers. This explains why the design is worth pursuing; it is an analytical result, separate from measured scaling.

Where the evidence is mixed. The family leads on the public benchmarks shown earlier, but does not lead everywhere. On P12 mortality it scores 0.871 versus 0.880 AUROC. On a second event benchmark, all seven overall results trail the published leaders, despite strong individual prediction components. Saved conventional language models also lead our character-level models; properly tokenized language development has beaten a classic count model. These are results for distinct tasks, not a single universal ranking.

The commercial significance. The public wins support the value of small, accurate models for timestamped data in industry, IT operations, security and finance. The learning work addresses a further source of cost: keeping models useful as new information arrives. If the measured component advantages carry through to complete applications, they could make continual adaptation more affordable on local machines and support sparse, clockless learning hardware. That broader opportunity is the significance of the design, rather than a measured revenue, energy or whole-model training claim.